Predicting Random-Walk Cover Times on Small Graphs from Simple Statistics

Anonymous Authors[1]

Abstract

The expected cover time of a simple random walk on a connected graph is classically bounded by $2m(n-1)$, a bound that is tight only for a few extremal graphs. For graphs with at most 30 vertices we ask whether a regression on cheap statistics (vertex count n, edge count m, maximum degree Δ, diameter D) predicts the cover time more accurately than this bound, and where the prediction fails. On 195 synthetic connected graphs from 13 families, measured by Monte Carlo with 1000 walks per graph, a log-linear model in $\log n, \log m, \log \Delta, \log D$ predicts held-out families with median multiplicative error $1.15\times$ (90th percentile $1.94\times$), versus $2.0\times$ ($6.0\times$) for the bound rescaled by a fitted constant and $3.9\times$ ($32\times$) for the raw bound. With four features the worst families are lollipops, complete graphs, binary trees and grids (about 1.7–2.0$\times$ median error); with two features the lollipop error rises to $5\times$. The results replicate across two seeds. This is a small, synthetic, single-start-distribution study, and we do not claim the fitted coefficients generalise beyond graphs of this size and family mix.

1. Introduction

Let $G = (V, E)$ be a connected undirected graph with n vertices and m edges. The cover time of the simple random walk is the expected number of steps needed to visit every vertex, starting from a given vertex, or averaged over starting vertices. Aleliunas et al. (Aleliunas et al., 1979) showed that it is at most $2m(n-1)$, a bound derived from commute times. Feige (Feige, 1995) showed that the worst case over n-vertex graphs is $(1 + o(1))\frac{4}{27}n^3$, achieved by lollipop-type graphs, and that the cover time is always at most this order. Surveys are given by Lovász (Lovász, 1993).

These are worst-case statements over all graphs of a given size. For a specific small graph they can be loose by one to two orders of magnitude: on the complete graph K_n the cover time is $\Theta(n \log n)$, while $2m(n-1) = \Theta(n^3)$. A practitioner who wants a number for a particular small graph would like a prediction that uses a handful of statistics that are trivial to compute.

We study the following question: for connected graphs with at most 30 vertices, can a few easily computed statistics give a more accurate prediction of the cover time than the general bound, and how does the error depend on the graph family? Our contribution is an empirical answer with explicit Monte Carlo uncertainty, a leave-one-family-out evaluation that tests transfer to unseen structure, and a per-family breakdown that shows where the predictor is reliable and where it is not.

2. Method

Target. For each graph we estimate $\mathbb{E}[\tau_{\text{cov}}]$ by simulating $R = 1000$ independent walks, each started at a vertex drawn uniformly at random, and recording the step at which the last vertex is first visited. We report the sample mean $\hat{C}(G)$ and its standard error. This averages over starting vertices rather than taking the maximum that defines the worst-case cover time; we return to this in Section 4.

Graphs. We generate even $n \in \{4, 6, \ldots, 30\}$ and the following families: path, cycle, star, complete, wheel, lollipop (clique of size $\lfloor n/2 \rfloor$ plus a path), barbell (two cliques joined by a path), near-square grid with $r \cdot c = n$, complete binary tree, uniform random trees (2 per n), and Erdős–Rényi $G(n, p)$ conditioned on connectivity for $p \in \{0.1, 0.25, 0.5\}$ (1 per n). This gives 195 graphs in total. The suite is a convenience sample, not a uniform sample over graphs of each size.

Features. For each graph we compute n, m, the maximum degree Δ and the diameter D by breadth-first search. The baseline quantity is $B(G) = 2m(n-1)$.

Predictors. We fit ordinary least squares to $\log \hat{C}(G)$ with two feature sets: (i) $\{1, \log n, \log m\}$ and (ii) $\{1, \log n, \log m, \log \Delta, \log D\}$. We also consider $c \cdot B(G)$ with c fitted as a geometric mean on all data, which is an

[1]Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

Preliminary work. Under review by the International Conference on Machine Learning (ICML). Do not distribute.

Table 1. Multiplicative prediction error over 195 graphs (median / 90th percentile). Lower is better. Seed 2026, $R = 1000$ walks per graph.

Predictor	Median	90th pct.
CV, features $\{n, m\}$	1.40	2.39
CV, features $\{n, m, \Delta, D\}$	**1.15**	**1.94**
$c \cdot 2m(n-1)$, c fitted in-sample	2.03	6.05
Raw bound $2m(n-1)$	3.86	32.4

in-sample and therefore optimistic baseline, and the raw bound $B(G)$.

Evaluation. We use leave-one-family-out cross-validation: each family is predicted by a model fitted on the other families, so the model is never tested on a structure it has seen. We report the median and 90th percentile of the multiplicative error $\exp|\log\hat{p} - \log\hat{C}|$ over all 195 graphs.

3. Experiments

All code is in `code/cover.py`. It uses only the Python standard library and runs in about two seconds on one CPU core with 1000 walks per graph. The main run uses seed 2026; a replicate uses seed 7.

Table 1 shows the main comparison. The four-feature model improves on the two-feature model and on both bound-based predictors, and its 90th percentile is much better than either bound. The raw bound is off by a median factor of 3.9 and by a factor of up to 32 on dense graphs. For the seed-7 replicate the medians are 1.39 (two features), 1.18 (four features), 2.10 (scaled bound) and 3.98 (raw bound); the 90th percentiles are within 5% of the main run. The fitted coefficients on the full data for the four-feature model are approximately $\log\hat{C} \approx -0.68 + 1.02\log n - 0.04\log m + 0.60\log\Delta + 1.14\log D$ (seed 2026) and $-0.69 + 0.99\log n - 0.02\log m + 0.61\log\Delta + 1.15\log D$ (seed 7). Because n, m, Δ and D are correlated within a family, the individual coefficients should not be read causally.

Table 2 shows that the error is strongly family-dependent. The 2-feature model is poor on lollipops (5.1×) and barbells (2.3×), where cover time is dominated by a bottleneck between dense and sparse regions that n and m cannot see. Adding Δ and D reduces the lollipop error to 1.8× and the barbell error to 1.1×. The 4-feature model is worst on complete graphs (2.0×), binary trees (1.7×) and grids (1.7×). Complete graphs lie far below the bound ($\hat{C}/B \approx 0.016$), so the fit must extrapolate in scale; binary trees and grids have diameters and degrees that vary little across our size range, so the regression has little signal to separate them from other families. Cycles, stars, random trees and sparse $G(n,p)$ are predicted within about 1.1×. Dense $G(n, 0.5)$ is at 1.4×, which suggests the statistics are less informative when cover time depends on many short routes rather than a single structure. Family-level errors are computed from only 2–20 graphs each, so they are indicative rather than precise.

Table 2. Per-family median multiplicative error under leave-one-family-out CV (seed 2026), for a selection of families, and the geometric mean of $\hat{C}/B$ (cover time divided by $2m(n-1)$).

Family	CV 4-feat.	CV 2-feat.	$\hat{C}/B$
Path	1.03	1.68	0.57
Cycle	1.09	1.40	0.25
Star	1.11	2.04	0.23
Barbell	1.09	2.26	0.27
Lollipop	1.79	5.07	0.38
Complete	1.98	1.82	0.016
Binary tree	1.68	1.14	0.42
Grid	1.65	1.56	0.14
Random tree	1.10	1.12	0.40
$G(n, 0.1)$	1.10	1.13	0.31
$G(n, 0.5)$	1.38	1.58	0.042

4. Discussion and Limitations

Where the predictor works. Across the families we tested, four cheap statistics give a predictor with a median error near 15%, which is substantially better than the classical bound. Families such as cycles, stars, random trees and sparse random graphs are predicted reliably.

Where it fails. Lollipops are the hardest family for the 2-feature model and remain the second-hardest for the 4-feature model; their cover time is set by a narrow bottleneck that (n, m, Δ, D) capture only indirectly. Complete graphs, binary trees and grids have the largest errors under the 4-feature model, and a user should not trust the predictor for those families without checking against simulation.

Limitations. (i) The target is the start-averaged cover time, not the maximum over starts; for lollipops these differ substantially, so our predictions are not directly comparable to Feige's worst-case constant. (ii) The graph suite is synthetic and sized by hand; it is not drawn from real networks, and the family mix strongly affects the fitted coefficients. (iii) Monte Carlo uncertainty is small relative to the reported errors: the largest standard error relative to the mean is 4% at $R = 1000$, and log-scale standard errors of this size do not change any conclusion. We did not produce a bootstrap confidence interval over graphs; the seed-2026 and seed-7 replicates are the only resampling evidence. (iv) Four features are a choice; we did not search over richer feature sets, so the predictor is a baseline, not an optimised one. (v)

Results are for $n \leq 30$; extrapolation to larger graphs is not supported.

5. Conclusion

For connected graphs with at most 30 vertices, a log-linear model in n, m, Δ and D predicts Monte Carlo cover times with a median error of about 15% under leave-one-family-out evaluation, compared with a median factor of about 4 for the raw bound $2m(n-1)$. The predictor is reliable on cycles, stars, barbells, random trees and sparse random graphs, and least reliable on lollipops, complete graphs, binary trees and grids. Reproducing these numbers requires only the included script.

Reproducibility

Environment: Python 3 standard library only (no numpy or networkx). Command: `python3 -I code/cover.py --out results --seed 2026 --trials 1000` and `--seed 7 --out results_seed7` for the replicate. Outputs: `graphs.csv` (per-graph statistics and Monte Carlo means with standard errors) and `report.json` (fitted coefficients and error summaries). Random seeds fix both graph generation and walk simulation. Runtime is about 2 seconds per run.

References

Aleliunas, R., Karp, R. M., Lipton, R. J., Lovász, L., and Rackoff, C. Random walks, universal traversal sequences, and the complexity of maze problems. In *20th Annual Symposium on Foundations of Computer Science (FOCS)*, pp. 218–223, 1979.

Feige, U. A tight upper bound on the cover time for random walks on graphs. *Random Structures & Algorithms*, 6(1): 51–54, 1995.

Lovász, L. Random walks on graphs: a survey. In *Combinatorics, Paul Erdős is Eighty*, volume 2, pp. 1–46. 1993.